

Employee Insurance Enrollment Prediction

1. Project Overview

The objective of this project is to build a machine learning pipeline that predicts whether an employee will enroll in a new voluntary insurance product based on demographic and employment-related information.

The project covers the complete machine learning workflow:

1. Loading and inspecting the employee dataset
 2. Data quality checks
 3. Feature preparation
 4. Numerical and categorical feature preprocessing
 5. Train/test splitting
 6. Random Forest model development
 7. Hyperparameter tuning
 8. Model evaluation
 9. Saving the best trained model
 10. Running predictions on new employee data
 11. Serving the trained model through a REST API using FastAPI
-

2. Dataset

The dataset provided for the assignment is:

`employee_data.csv`

It contains approximately 10,000 synthetic employee records.

The available columns are:

Column	Type	Description
<code>employee_id</code>	Numeric	Unique employee identifier
<code>age</code>	Numeric	Employee age

<code>gender</code>	Categorical	Employee gender
<code>marital_status</code>	Categorical	Marital status
<code>salary</code>	Numeric	Employee salary
<code>employment_type</code>	Categorical	Employment type
<code>region</code>	Categorical	Employee region
<code>has_dependents</code>	Categorical	Whether the employee has dependents
<code>tenure_years</code>	Numeric	Years with the company
<code>enrolled</code>	Binary	Target variable: 1 = enrolled, 0 = not enrolled

The target variable is:

`enrolled`

where:

- `1` = employee enrolled in the insurance product
- `0` = employee did not enroll

3. Data Observations

3.1 Dataset Size

The dataset contains approximately 10,000 records and 10 columns.

After removing the target column and employee identifier, the model uses 8 predictive features.

The dataset was split using an 80/20 stratified split:

- Training samples: **8,000**
- Testing samples: **2,000**

Stratification was used to preserve the target class distribution between the training and testing sets.

3.2 Feature Types

The numerical features identified in the dataset were:

- `age`
- `salary`
- `tenure_years`

The categorical features were:

- `gender`
 - `marital_status`
 - `employment_type`
 - `region`
 - `has_dependents`
-

3.3 Employee ID

`employee_id` was excluded from model training.

It is an identifier rather than a meaningful employee characteristic. Including it could allow the model to learn patterns associated with the ID rather than actual employee attributes.

3.4 Missing Values

Missing values were checked before training.

Numerical values, if missing, are handled using median imputation.

Categorical values, if missing, are handled using the most frequent category.

This preprocessing is included inside the machine learning pipeline so that preprocessing is consistently applied during both training and inference.

3.5 Synthetic Dataset

The dataset is synthetic rather than production employee data.

The resulting model performance is therefore interpreted in the context of the provided dataset and should not be assumed to represent real-world insurance enrollment performance.

4. Data Processing

The following preprocessing steps were used.

Numerical Features

The numerical features were:

- `age`
- `salary`
- `tenure_years`

Missing numerical values are replaced using median imputation.

Categorical Features

The categorical features were:

- `gender`
- `marital_status`
- `employment_type`
- `region`
- `has_dependents`

Categorical missing values are replaced with the most frequent category and categorical variables are converted to numerical representations using one-hot encoding.

`handle_unknown="ignore"` is used so that the model can handle an unseen category during inference.

Pipeline

The preprocessing and model are combined into a single scikit-learn Pipeline.

This ensures that the same preprocessing applied during training is automatically applied when making predictions on new employee records.

5. Model Choice

Random Forest

A Random Forest classifier was selected as the primary model.

Random Forest is suitable for this dataset because:

- The dataset is tabular.
- It contains both numerical and categorical features.
- One-hot encoding can easily represent the categorical variables.
- Random Forest can model non-linear relationships.
- It is relatively robust to feature scaling and different feature distributions.
- It provides probability estimates that can be used to estimate enrollment likelihood.
- It also provides feature importance information.

The initial model used:

- 300 trees
 - Balanced class weights
 - Fixed random state for reproducibility
 - Parallel processing using all available CPU cores
-

6. Hyperparameter Tuning

Hyperparameter tuning was added using [RandomizedSearchCV](#).

The parameters explored included:

- Number of trees
- Maximum tree depth
- Minimum samples required to split a node
- Minimum samples required at a leaf
- Number of features considered at each split

The tuning process used ROC-AUC as the optimization metric.

Three-fold cross-validation was used during the search.

The final model was selected based on the best cross-validation ROC-AUC score.

7. Evaluation Method

The final evaluation was performed on the held-out test set.

The following metrics were calculated:

- Accuracy
- Precision
- Recall
- F1 Score
- ROC-AUC

ROC-AUC was included because the model produces enrollment probabilities rather than only binary predictions.

The test set contained 2,000 samples.

8. Evaluation Results

The initial Random Forest model achieved the following results on the held-out test set:

Metric	Result
Accuracy	1.0000
Precision	1.0000
Recall	1.0000
F1 Score	1.0000
ROC-AUC	1.0000

Classification Report

	precision	recall	f1-score	support
0	1.00	1.00	1.00	765

	1	1.00	1.00	1.00	1235
accuracy			1.00	2000	
macro avg	1.00	1.00	1.00	2000	
weighted avg	1.00	1.00	1.00	2000	

Confusion Matrix

```
[[765  0]
 [ 0 1235]]
```

The model correctly classified all 2,000 test examples in the current evaluation.

9. Interpretation of Results

The model achieved perfect performance on the held-out test set.

This is an unusually strong result for a real-world enrollment prediction problem, but the dataset provided for this assignment is synthetic.

The perfect result suggests that the generated dataset contains highly separable patterns between enrolled and non-enrolled employees.

The results should therefore not be interpreted as evidence that an equivalent model would achieve 100% accuracy on real employee enrollment data.

For a production system, evaluation on a completely independent dataset collected at a later point in time would be important.

10. Model Persistence

After training, the best model is saved using `joblib`.

The saved model contains the complete preprocessing and classification pipeline.

The model is stored at:

`models/best_model.joblib`

This allows the exact trained model to be reused for inference without retraining.

11. Inference

A separate `inference.py` script was implemented to make predictions for individual employees.

The script accepts employee information such as:

- Age
- Gender
- Marital status
- Salary
- Employment type
- Region
- Dependents
- Tenure

The model returns:

- Enrollment probability
- Predicted enrollment class

For example:

Enrollment probability: 97.32%

Predicted enrollment: Yes

Predictions are also logged to:

reports/predictions.csv

This provides a record of inference requests and their resulting predictions.

12. REST API

A REST API was implemented using FastAPI.

The API loads the saved model from:

models/best_model.joblib

The main prediction endpoint is:

POST /predict

The API accepts employee information as JSON.

Example:

```
{
  "age": 35,
  "gender": "Male",
  "marital_status": "Married",
  "salary": 75000,
  "employment_type": "Full-time",
  "region": "West",
  "has_dependents": "Yes",
  "tenure_years": 8
}
```

The API returns the enrollment probability and predicted class.

Example response:

```
{
  "enrollment_probability": 0.9732,
  "predicted_enrollment": 1,
  "enrolled": "Yes"
}
```

FastAPI also automatically provides interactive API documentation at:

<http://127.0.0.1:8000/docs>

This makes the trained model accessible as a simple prediction service.

13. Experiment Tracking

A lightweight experiment log was implemented instead of adding a larger experiment tracking dependency.

The training script records model evaluation results in:

reports/experiments.csv

The saved metrics include:

- Accuracy
- Precision
- Recall
- F1
- ROC-AUC
- Training/tuning time

Detailed results and model configuration are also saved in:

reports/metrics.json

This provides reproducible records of the experiments performed during development.

14. Key Takeaways

1. The dataset contains a mixture of numerical and categorical employee features.
 2. `employee_id` was excluded because it is an identifier rather than a predictive feature.
 3. A preprocessing pipeline was used to handle numerical and categorical data consistently.
 4. Random Forest was selected because it works well for tabular data and can capture non-linear relationships.
 5. Hyperparameter tuning was added using `RandomizedSearchCV`.
 6. The model achieved 1.00 accuracy, precision, recall, F1, and ROC-AUC on the provided held-out test set.
 7. The perfect performance is likely related to the synthetic nature and separability of the provided dataset.
 8. The trained model was saved and reused for inference.
 9. A separate prediction script was implemented for individual employee predictions.
 10. A FastAPI REST service was added to expose the model through an API.
-

15. What I Would Do Next

With more time, I would extend the project in the following areas.

Model comparison

I would compare Random Forest with models such as:

- Logistic Regression
- Gradient Boosting
- HistGradientBoosting
- XGBoost or LightGBM if external dependencies were allowed

This would help determine whether the Random Forest is actually the best model.

Independent validation

I would evaluate the final model on a completely independent dataset.

This would provide a better estimate of how well the model generalizes.

Fairness analysis

Since demographic attributes are present, I would evaluate performance across relevant groups and check for significant differences in:

- False positive rates
- False negative rates
- Recall
- Precision

Production deployment

For a production system, I would containerize the FastAPI service, add request validation and logging, introduce monitoring, and establish model versioning.

16. Conclusion

The project implements a complete machine learning workflow for predicting voluntary insurance enrollment.

The final solution includes data preprocessing, Random Forest model training, hyperparameter tuning, evaluation, model persistence, individual prediction, experiment logging, and a FastAPI REST API.

The current model achieves perfect performance on the provided synthetic test set. While this demonstrates that the model successfully captures the patterns present in the supplied data, independent validation would be required before considering the model suitable for real-world employee insurance enrollment decisions.